

Abacus Romanus

Notatione Polonica

Theoretical background

Once upon a time in Roman times, aliens visited the earth and grabbed their numerals. Since they weren't satisfied with the numerical range, they added their own symbols. And since they also disliked parentheses, they stole modern Polish Notation as a system for writing expressions.

Goal

Your task is to create an abacus that uses Roman numerals and Polish Notation (PN) to record operations without parentheses. Don't confuse it with the even more famous Reverse Polish Notation (RPN). In addition to the ancient Roman numerals, the system is extended with several new ones such as F, P, G, A. In the Abacus you must implement the following basic arithmetic operations: addition '+', subtraction '-', and multiplication '*'. The arithmetic expression is entered into Abacus as an ASCII string, and the result should be an ASCII string, as well. The numbers are separated by either a space or an operator. Operators can be separated by space, as well.

Table 1 Extended Roman numerals.

Symbol	Value
A	5000
G	10000
P	50000
F	100000

In such a system, we have the correct range (in decimal): 1 – 399999. Neither zero nor negative are possible to represent.

If during calculation the result (also intermediate) is out of range, it means an error, and the Latin word for 'error' should be returned as the result to the Judge.

You can assume that the Judge only sends one correct expression to be calculated in one data block.

Table 2 Examples

Expression	Result
+I X	XI
- -X I V	IV

--XXX I- V I	XXV
*+I I X	XX
*X+I I	XX

Provided module

You can edit only the *task_12.sv* file. If you wish, you may add other files for supportive submodules.

The module has the following inputs and outputs:

Table 3 List of inputs and outputs of the task_2 module.

Signal name	Signal type	Bit length
i_clk	Input	1 bit
i_valid	Input	1 bit
i_rst	Input	1 bit
i_data	Input	8 bits
i_last	Input	1 bit
i_first	Input	1 bit
o_data	Output	8 bits
o_valid	Output	1 bit
o_last	Output	1 bit

The reset is synchronous and active high.

Input and output interfaces

In one packet the task receives a sequence of data on the *i_data* port. The maximal amount of data is 4KB. The beginning of the data block is marked by the *i_first* signal and the end by the *i_last* signal. Throughout the data transfer, the *i_valid* signal is high.

Task receives data in the way shown in the figure below:

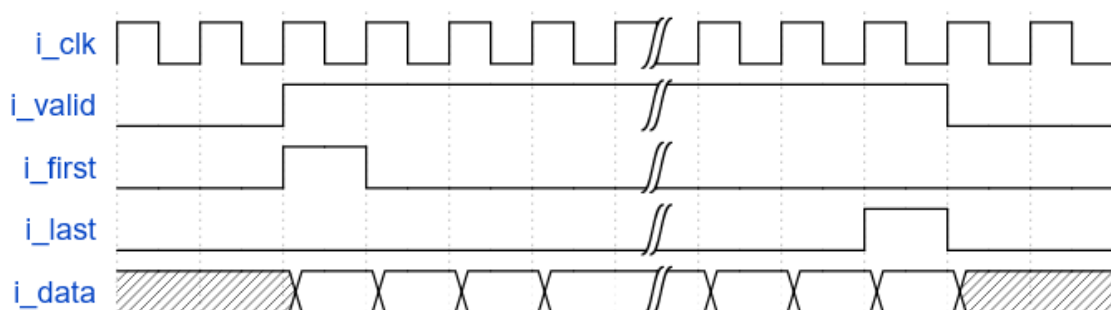


Figure 1 Input waveforms.

To send data from the task in a proper way, one needs to provide the *o_valid* signal that is synchronized with data on the *o_data* bus and *o_last* signal that is synchronized with the last sample in the output packet.

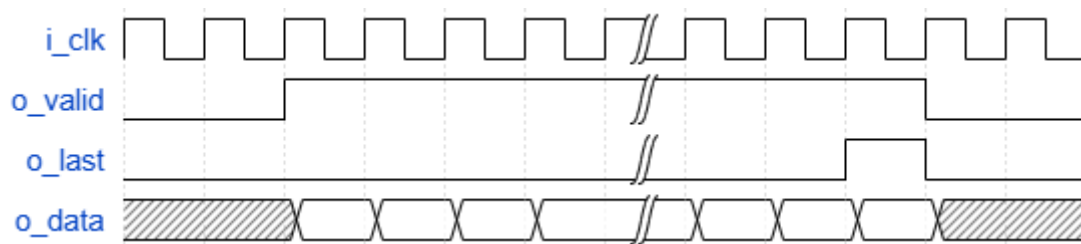


Figure 2 Output waveforms.

Output data can be sent as a continuous stream, or it can have regular or irregular gaps. The only requirement is providing *o_valid* and *o_last* signals.

Evaluating the task

The evaluation is done by comparing the sequence returned by the task with the expected sequence calculated by the reference model in the Judge. If the sequences are the same, the task is marked as correctly solved.

In Development mode, the task will be tested using a single set of data that is randomized on each run. In Evaluation mode, the task will be tested using 3 fixed sets of data, which remain the same across all runs and are identical for every team (for more information, check chapter 2.3.1 **Testing modes** in the **FPGA_Hackathon_2026_Project_Guide** document).

You earn 4 points for correctly processing each test vector. In the Development mode, this value simply indicates that the task passed the test. In the Evaluation mode, your total base score is calculated by multiplying the number of points by the number of test vectors (3), so you can earn up to 12 points.

Additional points (up to 5) may be awarded for **resource utilization when latency is below 10μs (723 cycles)** (for more information, check chapter 2.3.2 **Bonus Points** in the **FPGA_Hackathon_2026_Project_Guide** document).